

Name: Josh

1. Sailors-Boats-Reserves Database

Schema:

- Sailor(sid, sname, rating, age)
- Boats (bid, bname, color)
- Reserves (sid, bid, date)

1. Find the colors of boats reserved by Albert.

Relational Algebra

$$\pi_{\text{color}}(\sigma_{\text{sname}='Albert'}(\text{Sailor})\text{ReservesBoats})$$

SQL

```
SELECT DISTINCT B.color
FROM Sailor S
JOIN Reserves R ON S.sid = R.sid
JOIN Boats B ON R.bid = B.bid
WHERE S.sname = 'Albert';
```

2. Find all sailor IDs of sailors who have a rating of at least 8 or reserved boat 103.

Relational Algebra

$$\pi_{\text{sid}}(\sigma_{\text{rating} \geq 8}(\text{Sailor})) \cup \pi_{\text{sid}}(\sigma_{\text{bid}=103}(\text{Reserves}))$$

SQL

```
SELECT S.sid
FROM Sailor S
WHERE S.rating >= 8
UNION
SELECT R.sid
FROM Reserves R
WHERE R.bid = 103;
```

3. Find the names of sailors who have not reserved a red boat.

Relational Algebra

$$\pi_{\text{sname}}(\text{Sailor}) - \pi_{\text{sname}}(\sigma_{\text{color}='red'}(\text{Boats})\text{ReservesSailor})$$

SQL

```
SELECT S.sname
FROM Sailor S
WHERE S.sid NOT IN (
    SELECT R.sid
    FROM Reserves R
    JOIN Boats B ON R.bid = B.bid
    WHERE B.color = 'red'
);
```

4. Find the sailor IDs of sailors with age over 20 who have not reserved a red boat.

Relational Algebra

$$\pi_{\text{sid}}(\sigma_{\text{age} > 20}(\text{Sailor})) - \pi_{\text{sid}}(\sigma_{\text{color} = \text{'red'}}(\text{Boats})\text{Reserves})$$

SQL

```
SELECT S.sid
FROM Sailor S
WHERE S.age > 20
EXCEPT
SELECT R.sid
FROM Reserves R
JOIN Boats B ON R.bid = B.bid
WHERE B.color = 'red';
```

5. Find the names of sailors who have reserved all boats called BigBoat.

Relational Algebra

$$\pi_{\text{sname}}((\pi_{\text{sid}, \text{bid}}(\text{Reserves}) \div \pi_{\text{bid}}(\sigma_{\text{bname} = \text{'BigBoat'}}(\text{Boats})))\text{Sailor})$$

SQL

```
SELECT S.sname
FROM Sailor S
WHERE NOT EXISTS (
    -- Find a 'BigBoat'
    SELECT B.bid
    FROM Boats B
    WHERE B.bname = 'BigBoat'
    AND NOT EXISTS (
        -- ...that this sailor has NOT reserved
        SELECT R.bid
        FROM Reserves R
        WHERE R.bid = B.bid AND R.sid = S.sid
    )
);
```

2. University Database

Schema:

- Students (StudentID, FirstName, LastName, DepartmentID)
- Courses (CourseID, CourseName, InstructorID)
- Enrollments (EnrollmentID, StudentID, CourseID, Grade)
- Instructors (InstructorID, InstructorName, DepartmentID)
- Departments (DepartmentID, DepartmentName)

1. Retrieve the first and last names of all students.

Relational Algebra

$$\pi_{\text{FirstName, LastName}}(\text{Students})$$

SQL

```
SELECT FirstName, LastName
FROM Students;
```

2. List the names of students who are enrolled in the course with CourseID 101.

Relational Algebra

$$\pi_{\text{FirstName, LastName}}(\sigma_{\text{CourseID}=101}(\text{Enrollments})\text{Students})$$

SQL

```
SELECT DISTINCT S.FirstName, S.LastName
FROM Students S
JOIN Enrollments E ON S.StudentID = E.StudentID
WHERE E.CourseID = 101;
```

3. Find the names of instructors in the 'Computer Science' department.

Relational Algebra

$$\pi_{\text{InstructorName}}(\sigma_{\text{DepartmentName}='Computer Science'}(\text{Departments})\text{Instructors})$$

SQL

```
SELECT I.InstructorName
FROM Instructors I
JOIN Departments D ON I.DepartmentID = D.DepartmentID
WHERE D.DepartmentName = 'Computer Science';
```

4. List the students who have not enrolled in any course.

Relational Algebra

$$\pi_{\text{FirstName, LastName}}((\pi_{\text{StudentID}}(\text{Students}) - \pi_{\text{StudentID}}(\text{Enrollments}))\text{Students})$$

SQL

```
SELECT S.FirstName, S.LastName
FROM Students S
LEFT JOIN Enrollments E ON S.StudentID = E.StudentID
WHERE E.EnrollmentID IS NULL;
```

5. Find the StudentID, FirstName, and Last Name of students who have an 'A' grade in all their courses.

Relational Algebra

$$\pi_{\text{StudentID, FirstName, LastName}}((\pi_{\text{StudentID}}(\text{Enrollments}) - \pi_{\text{StudentID}}(\sigma_{\text{Grade} \neq 'A'}(\text{Enrollments})))\text{Students})$$

SQL

```
SELECT S.StudentID, S.FirstName, S.LastName
FROM Students S
WHERE S.StudentID IN (
    -- Students who are enrolled
    SELECT StudentID FROM Enrollments
    EXCEPT
    -- Minus students who have at least one non-'A' grade
    SELECT StudentID FROM Enrollments WHERE Grade <> 'A'
);
```

6. List the names of students who have received an 'A' grade in any course.

Relational Algebra

$$\pi_{\text{FirstName, LastName}}(\sigma_{\text{Grade}='A'}(\text{Enrollments})\text{Students})$$

SQL

```
SELECT DISTINCT S.FirstName, S.LastName
FROM Students S
JOIN Enrollments E ON S.StudentID = E.StudentID
WHERE E.Grade = 'A';
```

7. Find the StudentID, FirstName, and LastName of students who are taking either 'Computer Science' or 'Physics' courses.

Relational Algebra

$$\pi_{\text{StudentID, FirstName, LastName}}((\sigma_{\text{DepartmentName}='Computer Science' \text{ OR } \text{DepartmentName}='Physics'}(\text{Departments}))\text{InstructorsCourses})$$

SQL

```
SELECT DISTINCT S.StudentID, S.FirstName, S.LastName
FROM Students S
JOIN Enrollments E ON S.StudentID = E.StudentID
JOIN Courses C ON E.CourseID = C.CourseID
JOIN Instructors I ON C.InstructorID = I.InstructorID
JOIN Departments D ON I.DepartmentID = D.DepartmentID
WHERE D.DepartmentName = 'Computer Science' OR D.DepartmentName = 'Physics';
```

8. Retrieve the names of instructors who are not associated with any course.

Relational Algebra

$$\pi_{\text{InstructorName}}((\pi_{\text{InstructorID}}(\text{Instructors}) - \pi_{\text{InstructorID}}(\text{Courses}))\text{Instructors})$$

SQL

```
SELECT I.InstructorName
FROM Instructors I
LEFT JOIN Courses C ON I.InstructorID = C.InstructorID
WHERE C.CourseID IS NULL;
```

9. Find the names of students who have enrolled in both 'Math' and 'Physics' courses.

Relational Algebra

$$\text{MathStuds} \leftarrow \pi_{\text{StudentID}}((\sigma_{\text{DepartmentName}='Math'}(\text{Departments}))\text{InstructorsCoursesEnrollments})$$
$$\text{PhyStuds} \leftarrow \pi_{\text{StudentID}}((\sigma_{\text{DepartmentName}='Physics'}(\text{Departments}))\text{InstructorsCoursesEnrollments})$$
$$\pi_{\text{FirstName, LastName}}((\text{MathStuds} \cap \text{PhyStuds})\text{Students})$$

SQL

```
SELECT S.FirstName, S.LastName
FROM Students S
JOIN Enrollments E ON S.StudentID = E.StudentID
JOIN Courses C ON E.CourseID = C.CourseID
JOIN Instructors I ON C.InstructorID = I.InstructorID
JOIN Departments D ON I.DepartmentID = D.DepartmentID
WHERE D.DepartmentName IN ('Math', 'Physics')
GROUP BY S.StudentID, S.FirstName, S.LastName
HAVING COUNT(DISTINCT D.DepartmentName) = 2;
```

10. List the first and last names of all students who belong to the 'Computer Science' department.

Relational Algebra

$$\pi_{\text{FirstName, LastName}}(\sigma_{\text{DepartmentName}='Computer Science'}(\text{Departments})\text{Students})$$

SQL

```
SELECT S.FirstName, S.LastName
FROM Students S
JOIN Departments D ON S.DepartmentID = D.DepartmentID
WHERE D.DepartmentName = 'Computer Science';
```